

```
ai-automator-full-history.md+255
1 # AI Business Automator – Full Project History
2
3 **Written:** 2026-06-21
4 **Project:** AI Business Automator – Python-native replacement for 4 buggy n8n workflows
5 **Repo:** `~/home/openclaw/workspace/portfolio-projects/ai-automator/`
6 **Live:** `https://5lanxo-ai-automator.hf.space` (building)
7
8 ---
9
10 ## 1. The Origin – Why We Pivoted From n8n
11
12 Before the Python-native version, this project ran on **4 n8n workflows** – node-based
13 automation pipelines that handled email classification, inquiry responses, lead enrichment,
14 and daily reporting.
15
16 ### The Breaking Points
17
18 | Problem | Impact |
19 |-----|-----|
20 | **n8n Respond node couldn't evaluate `{{ }}` expressions** | Draft replies would render
21 raw template syntax instead of substituted values. Client-facing output was broken. |
22 | **Webhook timeouts** | n8n's webhook node had unreliable timeout behavior, especially
23 under free-tier hosting. Endpoints would hang or return 504. |
24 | **UI bugs in n8n editor** | Workflow nodes would misrender, connectors would break, and
25 the drag-and-drop interface introduced state corruption. |
26 | **No testability** | n8n workflows couldn't be unit-tested. Every change required manual
27 re-triggering through the UI. |
28 | **Monolithic workflow design** | All logic lived inside n8n's node graph. Version control
29 was nonexistent. |
30
31
32 **The Decision:** Replace all 4 workflows with a single FastAPI server. Same AI calls. Same
33 sidecar logic. Zero n8n UI bugs.
34
35 ---
36
37 ## 2. The Planning Phase
38
39 ### Documents Created
40
41 **BUILD_PLAN.md** – 11-task architectural spec with educational callouts explaining the
42 "why" behind each decision:
43
44 | Task | What Was Built | Concept Taught |
45 |-----|-----|-----|
46 | 1 | Project scaffold + venv | Environment setup |
47 | 2 | config.py – API key loader | Environment variables + secret management |
48 | 3 | server.py skeleton + /health | FastAPI basics, decorators, auto-docs |
49 | 4 | POST /classify – Email classifier | Prompt engineering (temp=0.1, constrained output) |
50
51 | 5 | POST /respond – Inquiry router | Why FastAPI works where n8n broke |
52 | 6 | POST /enrich – Lead enricher | Structured outputs from AI (constrained generation) |
53 | 7 | POST /report – Auto reporter | AI summarization, cron-triggered design |
54 | 8 | static/index.html – Demo frontend | Serving static files, portfolio showcase |
55 | 9 | run.sh, Dockerfile, README | Production polish + deployment |
56 | 10 | Smoke test all 4 endpoints | Verification |
57 | 11 | Deploy to HF Spaces | Multi-app Spaces, secrets, live demo |
58
59
60 **CLINE_INSTRUCTIONS.md** – Builder's brief specifying project location, port 8770, API key
61 management, and dark theme style reference.
62
63 ---
64
65 ## 3. Phase 1: Cline Builds The Foundation
66
67 Cline executed the 11-task plan. Each task was built in order:
68
69 ### Tasks 1-3: Scaffold, Config, Server
70
71 Python 3.12 venv created with FastAPI, uvicorn, httpx, and python-dotenv. Config module
72 loads OPENROUTER_API_KEY from ~/.hermes/.env via python-dotenv. FastAPI skeleton with
73 /health endpoint, CORS middleware, static file serving.
74
75 ### Tasks 4-7: The 4 Endpoints
76
77 | Endpoint | Input | Output | Model | Temp |
78 |-----|-----|-----|-----|-----|
79 | POST /classify | from_email, subject, body | classification (JOB/CLIENT/URGENT/NOISE) |
80 deepseek-chat | 0.1 |
81 | POST /respond | classification, from_email, subject, body | draft_reply | deepseek-chat |
82 0.3 |
83 | POST /enrich | company_name, contact_email, context | enriched_data (JSON) | deepseek-r1
84 | 0.2 |
85 | POST /report | date, emails, leads, tasks, notes | summary, metrics, recommendations
86 deepseek-chat | 0.2 |
87
88 ### Task 8: Dashboard
89
90 Dark-themed single-page app with input forms for all 4 endpoints, live JSON results,
91 loading states, and error handling.
92
93 ### Task 9: Deploy Config
94
95 Dockerfile, run.sh, and README created. Port 8770 local, reads $PORT env on HF Spaces.
96
97 ### Task 10: Smoke Test
98
99 All 4 endpoints passed – health, classify (JOB), respond (draft reply), enrich (structured
100 JSON), report (summary + metrics).
101
102 ---
103
104 ## 4. Phase 2: Sweet Jones P Refactors & Production-Hardens
105
106 After Cline's build, Sweet Jones P took over for production hardening.
107
108 ### run.sh Fixes
109
110 | Issue | Fix |
111 |-----|-----|
112 | Mixed venv/ and .venv/ paths | Standardized to .venv variable |
113 | No error handling | Added set -euo pipefail |
114 | Missing .env warning | Added user-friendly guidance |
115 | No $PORT support | Added env var for HF Spaces |
116
117 ### server.py Production Hardening
118
119 - Removed reload=True (was leaking watcher processes)
120 - Added CORS middleware
121 - Full Pydantic model layer (EmailClassifyRequest, etc.)
122 - Dedicated _call_openrouter() helper with error handling
123 - Config loaded per-request for hot-reload
124
125 ### Voice Config System (Key Feature)
126
127 Added GET /config and PUT /config endpoints. Config reads from client_config.json every
128 request – changes are live immediately. No restart needed.
129
130 ### Dashboard Redesign
131
132 Two-tab layout: Tools (4 endpoints) + Voice Config (live editor). Config Editor with fields
133 for business name, voice, sign-off, CTA, max words, never-mention list, and per-category
134 tone overrides.
135
136 ### Dockerfile Cleanup
137
138 Explicit COPY per file instead of COPY . . . Smaller layers, self-documenting.
139
140 ### DEV_LOG.md
141
142 Full build journal written from scratch – every decision, roadblock, and fix.
143
144 ---
145
146 ## 5. The Final Architecture
147
148 ...
149
150 Browser → FastAPI Server (port 8770) → OpenRouter Gateway → DeepSeek (chat/r1)
151
152                                     ↓
153                                     client_config.json
154                                     (live-editable)
155
156 ...
157
158 ### Tech Stack
159
160 | Layer | Technology |
161 |-----|-----|
162 | Framework | FastAPI (async) |
163 | HTTP Client | httpx (async) |
164 | LLM Gateway | OpenRouter |
165 | Classification | deepseek/deepseek-chat ($0.14/M) |
166 | Response Generation | deepseek/deepseek-chat |
167 | Lead Enrichment | deepseek/deepseek-r1 ($0.55/M) |
168 | Reporting | deepseek/deepseek-chat |
169 | Frontend | Vanilla HTML/CSS/JS |
170 | Deployment | Docker + HF Spaces |
171
172
173 ### Multi-Tenant Config System
174
175 Same codebase supports multiple clients by swapping client_config.json:
176
177 ...
178
179 client_A_config.json → "Professional corporate tone"
180 client_B_config.json → "Houston energy, direct"
181 client_C_config.json → "Warm, relationship-focused"
182
183 ...
184
185 ---
186
187 ## 6. The Deploy Process
188
189 **HF Space:** 5lanxo/ai-automator
190 **URL:** https://5lanxo-ai-automator.hf.space
191 **Status:** APP_STARTING (Docker build in progress, ~5-15 min first deploy)
192
193 ### Steps Completed
194
195 1. Space created with Docker SDK
196 2. OPENROUTER_API_KEY added as Space secret
197 3. All project files uploaded
198 4. Docker build initiated
199
200
201 ### Files Uploaded to HF
202
203 .gitattributes, .gitignore, DEV_LOG.md, Dockerfile, README.md, client_config.json,
204 config.py, requirements.txt, run.sh, server.py, static/index.html
205
206 ### Known Deploy Consideration
207
208 client_config.json persists in container filesystem only. On container restart (redeploy,
209 sleep/wake), config resets to the committed file. Future: swap to database-backed config.
210
211 ---
212
213 ## 7. Known Issues & Roadmap
214
215 | Issue | Priority | Detail |
216 |-----|-----|-----|
217 | HF Spaces cold start | Medium | Free tier sleeps after inactivity; first request ~30s
218 wake |
219 | R1 latency | Low | /enrich takes 30-90s; acceptable for batch, not real-time |
220 | Rate limiting | Low | No concurrency gate yet |
221 | Config persistence on HF | Low | Container FS only; redeploy = lost edits |
222
223
224 ### Future Items
225
226 1. Rate limiter (token-bucket pattern)
227 2. Database-backed config (SQLite)
228 3. Webhook receiver for direct email intake
229 4. Unit tests (pytest)
230 5. CI/CD pipeline
231 6. Multi-client dashboard
232 7. Dashboard auth
233
234 ---
235
236 ## 8. Asset Inventory
237
238 ### Files
239
240 | File | Purpose |
241 |-----|-----|
242 | config.py | API key loading |
243 | server.py | FastAPI app (7 endpoints) |
244 | static/index.html | Dark dashboard |
245 | client_config.json | Editable voice config |
246 | requirements.txt | Dependencies |
247 | Dockerfile | HF Spaces deploy |
248 | run.sh | One-command launcher |
249 | BUILD_PLAN.md | Architecture spec |
250 | CLINE_INSTRUCTIONS.md | Build instructions |
251 | DEV_LOG.md | Build journal |
252 | README.md | API docs + portfolio pitch |
253
254
255 ### Cost Breakdown
256
257 | Model | Input | Output | Use |
258 |-----|-----|-----|-----|
259 | deepseek-chat | $0.14/M | $0.28/M | classify, respond, report |
260 | deepseek-r1 | $0.55/M | $2.19/M | enrich (reasoning) |
261
262 ---
263
264 ## 9. Timeline Summary
265
266 ...
267
268 [Planning]
269 BUILD_PLAN.md written
270 CLINE_INSTRUCTIONS.md written
271 Project scaffold created
272
273 [Cline Builds – Phase 1]
274 Tasks 1-3: Scaffold, config, server skeleton
275 Tasks 4-7: classify, respond, enrich, report endpoints
276 Task 8: Dark-themed dashboard
277 Task 9: Dockerfile, run.sh, README
278 Task 10: Smoke tests pass
279
280
281 [Sweet Jones P – Phase 2 Refactors]
282 run.sh: Fixed venv, $PORT, strict mode
283 server.py: Production hardening, config endpoints, Pydantic
284 Dockerfile: Explicit file COPY, Python 3.12
285 Dashboard: Complete redesign with Config Editor tab
286 client_config.json: Finalized voice/character settings
287 DEV_LOG.md: Full build journal
288
289
290 [Deploy]
291 HF Space created at 5lanxo/ai-automator
292 Build started (APP_STARTING)
293 Monitoring for RUNNING status
294
295 ...
296
297 No newline at end of file
```